Module Interface Specification for Software Engineering

Team #1, Sanskrit Ciphers
Omar El Aref
Dylan Garner
Muhammad Umar Khan
Aswin Kuganesan
Yousef Shahin

April 7, 2026

1 Revision History

Date	Version	Notes
Nov. 13, 2025	1.0	Added Section 2: Symbols, Abbreviations and Acronyms.
Nov. 13, 2025	1.1	Added Section 3: Introduction.
Nov. 13, 2025	1.2	Added Section 4: Notation.
Jan. 18, 2026	1.3	Added Section 5: Module Decomposition.
Jan. 18, 2026	1.4	Added Sections 6–29: Module Interface Specifications for all modules.
Jan. 18, 2026	1.5	Added Reflection and References.
Apr. 6, 2026	1.6	Expanded the Section 2 exception glossary for consistency with the module specifications.
Apr. 6, 2026	1.7	Removed the unimplemented `ML.TextSim` module from the MIS to align the document with the codebase.
Apr. 6, 2026	1.8	Added explicit `pre:` and `post:` conditions to access routine semantics for stronger interface precision.

2 Symbols, Abbreviations and Acronyms

The following symbols, abbreviations, and acronyms are used throughout this Module Interface Specification (MIS):

Project-Specific

- **SMFRP** — Sanskrit Manuscript Fragment Reconstruction Platform
- **SRS** — System Requirements Specification
- **MG** — Module Guide
- **MIS** — Module Interface Specification

Layer & Module Prefixes

- **HW** — Hardware-Hiding Level
- **UI** — User Interface Layer
- **Svc** — Service Layer
- **ML** — Machine Learning Layer
- **Data** — Data Management Layer
- **Util** — Utility / Software Decision Level

UI / Web Terms

- **DOM** — Document Object Model
- **UI** — User Interface
- **HTML** — HyperText Markup Language
- **API** — Application Programming Interface
- **URL** — Uniform Resource Locator

Data Types & Structures

- **ID** — Identifier (e.g., ImageID, FragmentID)
- **MB** — Megabyte
- **JSON** — JavaScript Object Notation
- **HTTP** — Hypertext Transfer Protocol
- **REST** — Representational State Transfer

Exceptions / Errors

- **AggregationError** — Score aggregation or multi-model fusion failure
- **AuthError** — Authentication or credential verification failure
- **CatalogError** — Catalog association lookup or persistence failure
- **ConfigError** — Missing or invalid configuration value
- **ExportError** — Export generation or serialization failure
- **FragmentError** — Fragment creation, update, or deletion failure
- **InboxOverflowError** — Too many pending authentication or request items to accept another
- **InvalidFormatError** — Uploaded file format or payload structure is not supported
- **LayoutError** — Canvas placement or rotation violates layout constraints
- **MatchError** — Fragment matching service or result generation failure
- **ModelError** — Machine learning inference, preprocessing, or feature extraction failure
- **NetworkError** — Network communication or backend connectivity failure
- **NotFoundError** — Requested resource does not exist or is not accessible in context
- **PreprocError** — Preprocessing job creation or execution failure
- **ProjectError** — Project creation, update, or query failure
- **SearchError** — Search query validation or execution failure
- **ServiceError** — Service-layer orchestration or downstream dependency failure

- **SessionError** — Session or saved-layout persistence failure
- **StorageWriteError** / **StorageReadError** — File or object storage I/O failure
- **UIRenderError** — User interface rendering or display update failure
- **UploadError** — File upload transfer failure
- **UserError** — User record creation or update failure
- **ValidationError** — Input, request, or argument validation failure

Machine Learning Terms

- **OCR** — Optical Character Recognition
- **ML** — Machine Learning

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Introduction **1**

4 Notation **1**
4.1 Primitive Data Types . . . 2
4.2 Derived Data Types . . . 2
4.3 Interface and Parameter Conventions . . . 2

5 Module Decomposition **3**
5.1 Module Hierarchy . . . 3
5.2 Description of Levels . . . 4
5.2.1 Hardware-Hiding Level . . . 4
5.2.2 Behaviour-Hiding Level . . . 4
5.2.3 Software Decision Level . . . 4

6 MIS of HW.ImageStorage **5**
6.1 Module . . . 5
6.2 Uses . . . 5
6.3 Syntax . . . 5
6.3.1 Exported Constants . . . 5
6.3.2 Exported Access Programs . . . 5
6.4 Semantics . . . 5
6.4.1 State Variables . . . 5
6.4.2 Environment Variables . . . 5
6.4.3 Assumptions . . . 6
6.4.4 Access Routine Semantics . . . 6
6.4.5 Local Functions . . . 7

7 MIS of UI.Auth **7**
7.1 Module . . . 7
7.2 Uses . . . 7
7.3 Syntax . . . 7
7.3.1 Exported Constants . . . 7
7.3.2 Exported Access Programs . . . 8
7.4 Semantics . . . 8
7.4.1 State Variables . . . 8
7.4.2 Environment Variables . . . 8
7.4.3 Assumptions . . . 8
7.4.4 Access Routine Semantics . . . 8

7.4.5 Local Functions . . . 9

8 MIS of UI.Upload **9**
8.1 Module . . . 9
8.2 Uses . . . 10
8.3 Syntax . . . 10
8.3.1 Exported Constants . . . 10
8.3.2 Exported Access Programs . . . 10
8.4 Semantics . . . 10
8.4.1 State Variables . . . 10
8.4.2 Environment Variables . . . 10
8.4.3 Assumptions . . . 10
8.4.4 Access Routine Semantics . . . 11
8.4.5 Local Functions . . . 11

9 MIS of UI.Canvas **12**
9.1 Module . . . 12
9.2 Uses . . . 12
9.3 Syntax . . . 12
9.3.1 Exported Constants . . . 12
9.3.2 Exported Access Programs . . . 12
9.4 Semantics . . . 12
9.4.1 State Variables . . . 12
9.4.2 Environment Variables . . . 13
9.4.3 Assumptions . . . 13
9.4.4 Access Routine Semantics . . . 13
9.4.5 Local Functions . . . 14

10 MIS of UI.Search **15**
10.1 Module . . . 15
10.2 Uses . . . 15
10.3 Syntax . . . 15
10.3.1 Exported Constants . . . 15
10.3.2 Exported Access Programs . . . 15
10.4 Semantics . . . 15
10.4.1 State Variables . . . 15
10.4.2 Environment Variables . . . 16
10.4.3 Assumptions . . . 16
10.4.4 Access Routine Semantics . . . 16
10.4.5 Local Functions . . . 17

11 MIS of UI.SaveSession **17**
11.1 Module . . . 17
11.2 Uses . . . 17
11.3 Syntax . . . 17
11.3.1 Exported Constants . . . 17
11.3.2 Exported Access Programs . . . 18
11.4 Semantics . . . 18
11.4.1 State Variables . . . 18
11.4.2 Environment Variables . . . 18
11.4.3 Assumptions . . . 18
11.4.4 Access Routine Semantics . . . 18
11.4.5 Local Functions . . . 19

12 MIS of UI.LoadSession **19**
12.1 Module . . . 19
12.2 Uses . . . 19
12.3 Syntax . . . 20
12.3.1 Exported Constants . . . 20
12.3.2 Exported Access Programs . . . 20
12.4 Semantics . . . 20
12.4.1 State Variables . . . 20
12.4.2 Environment Variables . . . 20
12.4.3 Assumptions . . . 20
12.4.4 Access Routine Semantics . . . 20
12.4.5 Local Functions . . . 22

13 MIS of Svc.API **22**
13.1 Module . . . 22
13.2 Uses . . . 22
13.3 Syntax . . . 22
13.3.1 Exported Constants . . . 22
13.3.2 Exported Access Programs . . . 23
13.4 Semantics . . . 23
13.4.1 State Variables . . . 23
13.4.2 Environment Variables . . . 23
13.4.3 Assumptions . . . 23
13.4.4 Access Routine Semantics . . . 23
13.4.5 Local Functions . . . 25

14 MIS of Svc.AuthZ **25**
14.1 Module . . . 25
14.2 Uses . . . 26
14.3 Syntax . . . 26

14.3.1 Exported Constants . . . 26
14.3.2 Exported Access Programs . . . 26
14.4 Semantics . . . 26
14.4.1 State Variables . . . 26
14.4.2 Environment Variables . . . 26
14.4.3 Assumptions . . . 26
14.4.4 Access Routine Semantics . . . 27
14.4.5 Local Functions . . . 27

15 MIS of Svc.Session **28**
15.1 Module . . . 28
15.2 Uses . . . 28
15.3 Syntax . . . 28
15.3.1 Exported Constants . . . 28
15.3.2 Exported Access Programs . . . 28
15.4 Semantics . . . 28
15.4.1 State Variables . . . 28
15.4.2 Environment Variables . . . 28
15.4.3 Assumptions . . . 29
15.4.4 Access Routine Semantics . . . 29
15.4.5 Local Functions . . . 29

16 MIS of Svc.ProcOrch **30**
16.1 Module . . . 30
16.2 Uses . . . 30
16.3 Syntax . . . 30
16.3.1 Exported Constants . . . 30
16.3.2 Exported Access Programs . . . 30
16.4 Semantics . . . 30
16.4.1 State Variables . . . 30
16.4.2 Environment Variables . . . 30
16.4.3 Assumptions . . . 30
16.4.4 Access Routine Semantics . . . 31
16.4.5 Local Functions . . . 31

17 MIS of Svc.MatchOrch **31**
17.1 Module . . . 31
17.2 Uses . . . 32
17.3 Syntax . . . 32
17.3.1 Exported Constants . . . 32
17.3.2 Exported Access Programs . . . 32
17.4 Semantics . . . 32
17.4.1 State Variables . . . 32

17.4.2 Environment Variables . . . 32
17.4.3 Assumptions . . . 32
17.4.4 Access Routine Semantics . . . 32
17.4.5 Local Functions . . . 33

18 MIS of Svc.Search **33**
18.1 Module . . . 33
18.2 Uses . . . 33
18.3 Syntax . . . 33
18.3.1 Exported Constants . . . 33
18.3.2 Exported Access Programs . . . 34
18.4 Semantics . . . 34
18.4.1 State Variables . . . 34
18.4.2 Environment Variables . . . 34
18.4.3 Assumptions . . . 34
18.4.4 Access Routine Semantics . . . 34
18.4.5 Local Functions . . . 35

19 MIS of Data.FragmentStore **35**
19.1 Module . . . 35
19.2 Uses . . . 35
19.3 Syntax . . . 35
19.3.1 Exported Constants . . . 35
19.3.2 Exported Access Programs . . . 35
19.4 Semantics . . . 36
19.4.1 State Variables . . . 36
19.4.2 Environment Variables . . . 36
19.4.3 Assumptions . . . 36
19.4.4 Access Routine Semantics . . . 36
19.4.5 Local Functions . . . 37

20 MIS of Data.Catalog **37**
20.1 Module . . . 37
20.2 Uses . . . 37
20.3 Syntax . . . 37
20.3.1 Exported Constants . . . 37
20.3.2 Exported Access Programs . . . 38
20.4 Semantics . . . 38
20.4.1 State Variables . . . 38
20.4.2 Environment Variables . . . 38
20.4.3 Assumptions . . . 38
20.4.4 Access Routine Semantics . . . 38
20.4.5 Local Functions . . . 39

21 MIS of Data.User **39**
21.1 Module . . . 39
21.2 Uses . . . 39
21.3 Syntax . . . 39
21.3.1 Exported Constants . . . 39
21.3.2 Exported Access Programs . . . 40
21.4 Semantics . . . 40
21.4.1 State Variables . . . 40
21.4.2 Environment Variables . . . 40
21.4.3 Assumptions . . . 40
21.4.4 Access Routine Semantics . . . 40
21.4.5 Local Functions . . . 41

22 MIS of Data.Project **41**
22.1 Module . . . 41
22.2 Uses . . . 42
22.3 Syntax . . . 42
22.3.1 Exported Constants . . . 42
22.3.2 Exported Access Programs . . . 42
22.4 Semantics . . . 42
22.4.1 State Variables . . . 42
22.4.2 Environment Variables . . . 42
22.4.3 Assumptions . . . 42
22.4.4 Access Routine Semantics . . . 42
22.4.5 Local Functions . . . 43

23 MIS of ML.EdgeMatch **44**
23.1 Module . . . 44
23.2 Uses . . . 44
23.3 Syntax . . . 44
23.3.1 Exported Constants . . . 44
23.3.2 Exported Access Programs . . . 44
23.4 Semantics . . . 44
23.4.1 State Variables . . . 44
23.4.2 Environment Variables . . . 44
23.4.3 Assumptions . . . 44
23.4.4 Access Routine Semantics . . . 45
23.4.5 Local Functions . . . 45

24 MIS of ML.ScriptClass **45**
24.1 Module . . . 45
24.2 Uses . . . 45
24.3 Syntax . . . 46

24.3.1 Exported Constants . . . 46
24.3.2 Exported Access Programs . . . 46
24.4 Semantics . . . 46
24.4.1 State Variables . . . 46
24.4.2 Environment Variables . . . 46
24.4.3 Assumptions . . . 46
24.4.4 Access Routine Semantics . . . 46
24.4.5 Local Functions . . . 46

25 MIS of ML.TextSim **47**
25.1 Module . . . 47
25.2 Uses . . . 47
25.3 Syntax . . . 47
25.3.1 Exported Constants . . . 47
25.3.2 Exported Access Programs . . . 47
25.4 Semantics . . . 47
25.4.1 State Variables . . . 47
25.4.2 Environment Variables . . . 47
25.4.3 Assumptions . . . 47
25.4.4 Access Routine Semantics . . . 48
25.4.5 Local Functions . . . 48

26 MIS of ML.Segmentation **48**
26.1 Module . . . 48
26.2 Uses . . . 48
26.3 Syntax . . . 48
26.3.1 Exported Constants . . . 48
26.3.2 Exported Access Programs . . . 49
26.4 Semantics . . . 49
26.4.1 State Variables . . . 49
26.4.2 Environment Variables . . . 49
26.4.3 Assumptions . . . 49
26.4.4 Access Routine Semantics . . . 49
26.4.5 Local Functions . . . 50

27 MIS of ML.CircleRecognition **50**
27.1 Module . . . 50
27.2 Uses . . . 50
27.3 Syntax . . . 50
27.3.1 Exported Constants . . . 50
27.3.2 Exported Access Programs . . . 51
27.4 Semantics . . . 51
27.4.1 State Variables . . . 51

27.4.2 Environment Variables . . . 51
27.4.3 Assumptions . . . 51
27.4.4 Access Routine Semantics . . . 51
27.4.5 Local Functions . . . 52

28 MIS of ML.LineCount **52**
28.1 Module . . . 52
28.2 Uses . . . 52
28.3 Syntax . . . 52
28.3.1 Exported Constants . . . 52
28.3.2 Exported Access Programs . . . 52
28.4 Semantics . . . 53
28.4.1 State Variables . . . 53
28.4.2 Environment Variables . . . 53
28.4.3 Assumptions . . . 53
28.4.4 Access Routine Semantics . . . 53
28.4.5 Local Functions . . . 53

29 MIS of ML.EdgeClassification **54**
29.1 Module . . . 54
29.2 Uses . . . 54
29.3 Syntax . . . 54
29.3.1 Exported Constants . . . 54
29.3.2 Exported Access Programs . . . 54
29.4 Semantics . . . 54
29.4.1 State Variables . . . 54
29.4.2 Environment Variables . . . 54
29.4.3 Assumptions . . . 55
29.4.4 Access Routine Semantics . . . 55
29.4.5 Local Functions . . . 55

30 MIS of Util.Error **55**
30.1 Module . . . 55
30.2 Uses . . . 56
30.3 Syntax . . . 56
30.3.1 Exported Constants . . . 56
30.3.2 Exported Access Programs . . . 56
30.4 Semantics . . . 56
30.4.1 State Variables . . . 56
30.4.2 Environment Variables . . . 56
30.4.3 Assumptions . . . 56
30.4.4 Access Routine Semantics . . . 56

31 MIS of Util.Logging **57**
31.1 Module . . . 57
31.2 Uses . . . 57
31.3 Syntax . . . 57
31.3.1 Exported Constants . . . 57
31.3.2 Exported Access Programs . . . 57
31.4 Semantics . . . 57
31.4.1 State Variables . . . 57
31.4.2 Environment Variables . . . 58
31.4.3 Assumptions . . . 58
31.4.4 Access Routine Semantics . . . 58

32 MIS of Util.Config **59**
32.1 Module . . . 59
32.2 Uses . . . 59
32.3 Syntax . . . 59
32.3.1 Exported Constants . . . 59
32.3.2 Exported Access Programs . . . 59
32.4 Semantics . . . 59
32.4.1 State Variables . . . 59
32.4.2 Environment Variables . . . 59
32.4.3 Assumptions . . . 59
32.4.4 Access Routine Semantics . . . 60

33 MIS of Util.TestStub **60**
33.1 Module . . . 60
33.2 Uses . . . 60
33.3 Syntax . . . 60
33.3.1 Exported Constants . . . 60
33.3.2 Exported Access Programs . . . 61
33.4 Semantics . . . 61
33.4.1 State Variables . . . 61
33.4.2 Environment Variables . . . 61
33.4.3 Assumptions . . . 61
33.4.4 Access Routine Semantics . . . 61
33.4.5 Local Functions . . . 62

List of Tables

1 Primitive Data Types . . . 2
2 Module Hierarchy . . . 4
3 HW.ImageStorage - Exported Access Programs . . . 5
4 UI.Auth - Exported Access Programs . . . 8
5 UI.Upload - Exported Access Programs . . . 10
6 UI.Canvas - Exported Access Programs . . . 12
7 UI.Search - Exported Access Programs . . . 15
8 UI.SaveSession - Exported Access Programs . . . 18
9 UI.LoadSession - Exported Access Programs . . . 20
10 Svc.API - Exported Access Programs . . . 23
11 Svc.AuthZ - Exported Access Programs . . . 26
12 Svc.Session - Exported Access Programs . . . 28
13 Svc.ProcOrch - Exported Access Programs . . . 30
14 Svc.MatchOrch - Exported Access Programs . . . 32
15 Svc.Search - Exported Access Programs . . . 34
16 Data.FragmentStore - Exported Access Programs . . . 35
17 Data.Catalog - Exported Access Programs . . . 38
18 Data.User - Exported Access Programs . . . 40
19 Data.Project - Exported Access Programs . . . 42
20 ML.EdgeMatch - Exported Access Programs . . . 44
21 ML.ScriptClass - Exported Access Programs . . . 46
22 ML.TextSim - Exported Access Programs . . . 47
23 ML.Segmentation - Exported Access Programs . . . 49
24 ML.CircleRecognition - Exported Access Programs . . . 51
25 ML.LineCount - Exported Access Programs . . . 52
26 ML.EdgeClassification - Exported Access Programs . . . 54
27 Util.Error - Exported Access Programs . . . 56
28 Util.Logging - Exported Access Programs . . . 57
29 Util.Config - Exported Access Programs . . . 59
30 Util.TestStub - Exported Access Programs . . . 61

3 Introduction

The following document details the Module Interface Specifications (MIS) for the Sanskrit Manuscript Fragment Reconstruction Platform (SMFRP). This system supports researchers and scholars in reconstructing fragmented Sanskrit manuscripts by providing a unified digital workspace that integrates fragment management, intelligent matching, and image-based reconstruction tools.

The MIS defines the external behavior and interfaces for each software module in the system. Each module is specified in terms of its responsibilities, provided services, hidden state (secrets), environment variables, and exported access programs. These specifications are written at the right level of abstraction, focusing on logical interfaces and responsibilities rather than low-level implementation details such as programming language syntax or data structures.

This document ensures that the system achieves consistency, and minimal coupling across modules. The defined interfaces promote high cohesion, and generality, enabling independent development, verification, and maintenance of each module.

Complementary Documents

- **System Requirements Specification (SRS)** – Defines the system's functional and non-functional requirements. The SRS (Ciphers, 2025) can be accessed at: SRS
- **Module Guide (MG)** – Outlines the overall software architecture, module decomposition, and dependency hierarchy.

Repository

The full project documentation and implementation can be found in the team's GitHub repository

4 Notation

In this section, we define the notation used throughout the Module Interface Specification. The purpose of this notation is to describe, in a clear and consistent manner, what each module provides and requires, without referencing any specific programming language or implementation detail.

Each module in this MIS is described in terms of:

- **State Variables** – The module's internal data that is not visible externally.
- **Environment Variables** – Data received from or affecting the external environment (e.g., databases, file storage, APIs).

- **Exported Constants and Access Programs** – The operations, services, or functions the module provides to other modules.

This approach focuses on interface behavior, not implementation syntax (such as JSON or Python code), ensuring that the interfaces remain consistent.

4.1 Primitive Data Types

Table 1: Primitive Data Types

Data Type	Notation	Description
Character	`char`	A single symbol or letter.
Integer	`int`	Whole numbers with no fractional part.
Real	`float`	Numbers with decimal or fractional parts.
Boolean	`bool`	Represents logical truth values (true or false).

4.2 Derived Data Types

In addition to the primitive types above, the system specification also makes use of the following derived or composite types:

- **Sequence:** An ordered collection of elements of the same data type. Example: A list of fragment identifiers or saved user sessions.
- **String:** A sequence of characters, typically used for textual input, labels, or metadata fields.
- **Tuple:** An ordered collection of values that may be of different data types. Commonly used to represent grouped entities such as `(FragmentId, ConfidenceScore)` or `(x, y, width, height)`.
- **Record / Object:** A structured type that groups named fields with defined data types, such as `{id: FragmentId, script: String, confidence: Float}`.
- **Function:** A mapping defined by the types of its inputs and outputs. Functions are specified by their type signatures (inputs/outputs) and behavior (preconditions, postconditions, and effects).

4.3 Interface and Parameter Conventions

This section explains how interfaces and operations are written in the MIS. The goal is to make them easy to read and understand while staying general enough to apply to any implementation.

- **Inputs and Outputs:** Labeled as `in:` and `out:` in operation definitions. Example: `search(in: SearchQuery, out: SearchResult)`
- **Operation Format:** `functionName(in: InputType, out: OutputType) → may throw ErrorType`
- **Module Naming Convention:** Each module name follows the format `Layer.Module`, for example: `UI.Canvas`, `Svc.MatchOrch`, or `Data.FragmentStore`.
- **Logical Expressions:** Represented using logical symbols such as $\wedge$ (and), $\vee$ (or), $\neg$ (not), and the implication operator $\implies$.

5 Module Decomposition

The following section summarizes the module hierarchy for the *Sanskrit Manuscript Fragment Reconstruction Platform*. It is consistent with the **Module Guide (MG)** for this project and represents how the system has been decomposed into modules at different abstraction levels.

Each level in the hierarchy captures a different design concern:

- **Hardware-Hiding Level:** Interfaces directly with the system's physical or platform-dependent resources.
- **Behaviour-Hiding Level:** Encapsulates the main functional behavior of the system, implementing the use cases described in the SRS.
- **Software Decision Level:** Contains lower-level utility modules and software decisions that simplify implementation but are not visible to the user.

5.1 Module Hierarchy

Table 2: Module Hierarchy

Level 1	**Level 2 Modules**
Hardware-Hiding (HW)	–
Behaviour-Hiding	`HW.ImageStorage` **User Interface (UI):** `UI.Canvas`, `UI.Search`, `UI.Auth`, `UI.Upload`, `UI.SaveSession`, `UI.LoadSession` **Service Layer (Svc):** `Svc.API`, `Svc.Search`, `Svc.AuthZ`, `Svc.Session`, `Svc.ProcOrch`, `Svc.MatchOrch` **Data Management (Data):** `Data.FragmentStore`, `Data.Catalog`, `Data.User`, `Data.Project` **Machine Learning Modules (ML):** `ML.EdgeMatch`, `ML.ScriptClass`, `ML.Segmentation`, `ML.CircleRecognition`, `ML.LineCount`, `ML.EdgeClassification`
Software Decision	Utility and Support Modules (Error Handling, Logging, Configuration Management, Test Stubs)

5.2 Description of Levels

5.2.1 Hardware-Hiding Level

This level hides operating-system and platform dependencies from the rest of the system. In this design, those low-level details remain external to the application modules rather than being specified as separate MIS leaf modules.

5.2.2 Behaviour-Hiding Level

This level defines the functional behavior of the system, implementing the requirements outlined in the SRS. It includes `HW.ImageStorage`, the user-facing modules (UI), the application logic (Service Layer), data persistence (Data Layer), and intelligent processing (ML Layer). These modules interact through clearly defined service interfaces, promoting modularity and separation of concerns.

5.2.3 Software Decision Level

This level includes supporting components that are primarily design decisions rather than functional necessities. These modules include configuration handlers, utility libraries, and testing or logging frameworks that support development, debugging, and maintenance.

6 MIS of HW.ImageStorage

6.1 Module

HW.ImageStorage (Image Storage Interface)

6.2 Uses

- External file system or object storage API.
- `Util.Error` – for constructing storage-related error objects.

6.3 Syntax

6.3.1 Exported Constants

- `MAX_FILE_SIZE = 50MB`
- `SUPPORTED_FORMATS = {.jpg, .png, .tiff}`
- `STORAGE_PATH = "/images/"`

6.3.2 Exported Access Programs

Table 3: HW.ImageStorage - Exported Access Programs

Name	In	Out	Exceptions
storeImage	file: ImageFile, metadata: Record	ImageID	InvalidFormatError, StorageWriteError
retrieveImage	id: ImageID	ImageFile	NotFoundError, StorageReadError
deleteImage	id: ImageID	bool	NotFoundError, StorageWriteError
getMetadata	id: ImageID	Record	NotFoundError

6.4 Semantics

6.4.1 State Variables

- `storageIndex: Map[ImageID, FilePath]`
- `metadataCache: Map[ImageID, Record]`

6.4.2 Environment Variables

- `FileSystem / CloudStorage` – external persistence for image binaries

6.4.3 Assumptions

- The underlying storage service guarantees atomic read/write operations.

6.4.4 Access Routine Semantics

`storeImage(file, metadata)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: a candidate `ImageID` is generated and checked with `isUniqueImageID`; if the identifier is unused, the file is written to storage and `storageIndex` and `metadataCache` are updated with the new entry.
- output: a newly generated unique `ImageID`.
- exception: `InvalidFormatError` if file type not in `SUPPORTED_FORMATS`; `StorageWriteError` if write fails.

`retrieveImage(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: image file associated with `id`.
- exception: `NotFoundError` if `id` absent from `storageIndex`; `StorageReadError` if read fails.

`deleteImage(id)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: remove image file from storage; delete entries from `storageIndex` and `metadataCache`.
- output: `true` if deletion successful.

- exception: `NotFoundError` if `id` not present; `StorageWriteError` if deletion fails.

`getMetadata(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: metadata record associated with `id`.
- exception: `NotFoundError` if no record exists.

6.4.5 Local Functions

- `generateImageID(file: ImageFile): ImageID` – generates a candidate identifier for a newly stored image.
- `isUniqueImageID(id: ImageID): bool` – returns `true` iff `id` is not already present in `storageIndex`.
- `validateFormat(file: ImageFile): bool` – checks that the uploaded file format is supported.

7 MIS of UI.Auth

7.1 Module

UI.Auth (Authentication Interface)

This module provides the browser-facing login and logout experience. It collects user credentials, displays relevant error messages, and forwards authentication requests to the backend service. It does not perform authentication logic itself.

7.2 Uses

- `Svc.AuthZ` – for validating credentials and issuing session tokens.
- `Svc.Session` – for managing client-side session lifecycle.

7.3 Syntax

7.3.1 Exported Constants

- `MAX_LOGIN_ATTEMPTS = 5`

7.3.2 Exported Access Programs

Table 4: UI.Auth - Exported Access Programs

Name	In	Out	Exceptions
`renderLoginForm`	`none`	`none`	`UIRenderError`
`submitCredentials`	`creds: UserCredentials`	`AuthResult`	`AuthError, NetworkError, InboxOverflowError`
`logoutUser`	`none`	`bool`	`NetworkError`

7.4 Semantics

7.4.1 State Variables

- `loginAttempts: int` – count of consecutive failed attempts in current session.
- `authErrorMessage: String` – message shown on failed login.

7.4.2 Environment Variables

- Browser window and Document Object Model(DOM).

7.4.3 Assumptions

- Password input is masked and handled securely by the browser.

7.4.4 Access Routine Semantics

`renderLoginForm()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: updates DOM to show login form, clears previous error message.
- output: none.
- exception: `UIRenderError` if DOM cannot be updated.

`submitCredentials(creds)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.

- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: checks whether `Svc.AuthZ` is reachable, then sends credentials to it; updates `loginAttempts` and `authErrorMessage` based on response.
- output: `AuthResult` indicating success or failure.
- exception: `AuthError` if credentials rejected; `NetworkError` if the reachability check fails or the backend is unreachable.

`logoutUser()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: clears client-side session state and redirects to login view.
- output: `true` if logout succeeds.
- exception: `NetworkError` if logout call to backend fails.

7.4.5 Local Functions

- `displayError(msg: String)` – updates UI error region.
- `checkAuthServiceReachable(): bool` – checks whether the authentication service can be contacted before submitting credentials.
- `resetForm()` – clears form fields.

8 MIS of UI.Upload

8.1 Module

UI.Upload (Fragment Upload Interface)

This module allows users to select, preview, and submit manuscript fragment images for processing. It performs basic client-side validation and delegates storage and preprocessing to backend services.

8.2 Uses

- `Svc.API` – specifically `POST /fragments/upload` for submitting image files and receiving fragment identifiers, and `GET /fragments/{id}/status` for polling processing status after upload.
- `HW.ImageStorage` – indirectly, via backend, for persistent image storage.

8.3 Syntax

8.3.1 Exported Constants

- `MAX_FILES_PER_UPLOAD = 20`

8.3.2 Exported Access Programs

Table 5: UI.Upload - Exported Access Programs

Name	In	Out	Exceptions
renderUploadPanel	none	none	UIRenderError
selectFiles	files: Sequence[ImageFile]	none	ValidationError
submitUpload	none	UploadResult	NetworkError, UploadError

8.4 Semantics

8.4.1 State Variables

- `pendingFiles: Sequence[ImageFile]`
- `uploadErrorMessage: String`

8.4.2 Environment Variables

- Browser file picker and local file system access (sandboxed).

8.4.3 Assumptions

- The browser enforces configured file size constraints on selection when possible.

8.4.4 Access Routine Semantics

`renderUploadPanel()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: renders the upload panel, including file input control and status indicators.
- output: none.
- exception: `UIRenderError` if DOM cannot be updated.

`selectFiles(files)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: validates file count and basic properties; if valid, stores them in `pendingFiles`.
- output: none.
- exception: `ValidationError` if any file exceeds size or format constraints.

`submitUpload()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: sends `pendingFiles` to backend via `Svc.API`; clears `pendingFiles` on success.
- output: `UploadResult` containing list of created fragment IDs.
- exception: `NetworkError` if request fails; `UploadError` if backend rejects the upload.

8.4.5 Local Functions

- `validateFiles(files)` – checks count, format, and size bounds.
- `previewThumbnails()` – generates and displays thumbnails for selected files.

9 MIS of UI.Canvas

9.1 Module

UI.Canvas (Interactive Fragment Workspace)

This module provides the interactive canvas where scholars can view, move, rotate, and arrange multiple fragments simultaneously. It displays match suggestions and visual overlays for edges, damage patterns, or text regions.

9.2 Uses

- `Svc.MatchOrch` – for requesting match suggestions.
- `Svc.Session` – for loading and saving canvas layouts.
- `Svc.API` – for fetching fragment images and overlays.

9.3 Syntax

9.3.1 Exported Constants

- `MAX_FRAGMENTS_ON_CANVAS = 50`

9.3.2 Exported Access Programs

Table 6: UI.Canvas - Exported Access Programs

Name	In	Out	Exceptions
renderCanvas	state: CanvasState	none	UIRenderError
placeFragment	id: FragmentID, position: Record{x: float, y: float}	none	LayoutError
rotateFragment	id: FragmentID, angle: float	none	LayoutError
requestMatches	id: FragmentID	none	NetworkError
saveLayout	none	LayoutID	NetworkError

9.4 Semantics

9.4.1 State Variables

- `currentState: CanvasState` – positions, rotations, and visibility for fragments.
- `selectedFragment: FragmentID or null`

- `activeMatchOverlay: MatchResultList`

9.4.2 Environment Variables

- Browser rendering surface (HTML canvas or equivalent).

9.4.3 Assumptions

- Device supports required rendering capabilities for zoom and pan.

9.4.4 Access Routine Semantics

`renderCanvas(state)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: updates the visual layout to match `state`.
- output: none.
- exception: `UIRenderError` if rendering fails.

`placeFragment(id, position)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: validates the `position` record, where `position.x` and `position.y` are the target canvas coordinates for the fragment's anchor point, then updates `currentState` with that new position for `id`.
- output: none.
- exception: `LayoutError` if fragment exceeds bounds or limit exceeded.

`rotateFragment(id, angle)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.

- transition: updates rotation of `id` in `currentState`.
- output: none.
- exception: `LayoutError` if rotation cannot be applied.

`requestMatches(id)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: sends match request to `Svc.MatchOrch`; updates `activeMatchOverlay` when response is received.
- output: none.
- exception: `NetworkError` if request fails.

`saveLayout()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: persists `currentState` via `Svc.Session`.
- output: a `LayoutID` for later retrieval.
- exception: `NetworkError` if persistence fails.

9.4.5 Local Functions

- `validateCanvasPosition(position: Record{x: float, y: float}): bool` – checks that a proposed placement is inside the canvas bounds.
- `snapToEdges(id)` – adjusts position so nearby edges align visually.
- `highlightMatches(id)` – visually emphasizes suggested neighbors.

10 MIS of UI.Search

10.1 Module

UI.Search (Database Search and Exploration Interface)

This module provides the interface for querying the fragment database, filtering results, and viewing metadata. It supports search by script type, line count, fragment size, and other criteria.

10.2 Uses

- `Svc.Search` – for executing search queries.
- `Svc.API` – for retrieving fragment summaries and export payloads.

10.3 Syntax

10.3.1 Exported Constants

- `RESULTS_PAGE_SIZE = 25`

10.3.2 Exported Access Programs

Table 7: UI.Search - Exported Access Programs

Name	In	Out	Exceptions
`renderSearchPanel`	`none`	`none`	`UIRenderError`
`submitQuery`	`q: SearchQuery`	`none`	`ValidationError, NetworkError`
`nextPage`	`none`	`none`	`NetworkError`
`exportResults`	`format: ExportFormat`	`none`	`NetworkError, ExportError`

10.4 Semantics

10.4.1 State Variables

- `currentQuery: SearchQuery`
- `currentPage: int`
- `results: SearchResultPage`

10.4.2 Environment Variables

- Browser viewport for displaying tables and filters.

10.4.3 Assumptions

- Backend search service responds within acceptable time bounds defined in the SRS.

10.4.4 Access Routine Semantics

`renderSearchPanel()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: displays search form, filters, and an empty or last-used results view.
- output: none.
- exception: `UIRenderError` if rendering fails.

`submitQuery(q)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: validates query parameters; if valid, sends query to `Svc.Search` and updates `results` and `currentPage`.
- output: none.
- exception: `ValidationError` if query malformed; `NetworkError` if backend unreachable.

`nextPage()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: increments `currentPage` and requests next result page.

- output: none.
- exception: `NetworkError` if request fails.

`exportResults(format)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: sends export request for current query to backend.
- output: none at UI level; triggers file download in browser.
- exception: `NetworkError` if backend unreachable; `ExportError` if export fails.

10.4.5 Local Functions

- `renderResultsTable(results)` – updates DOM table from latest `SearchResultPage`.
- `validateSearchQuery(q)` – checks filter ranges and required fields.

11 MIS of UI.SaveSession

11.1 Module

UI.SaveSession (Session Save Interface)

This module provides the user interface for saving the current workspace state, including fragment arrangements, annotations, and reconstruction progress. It allows users to name and persist their work for later retrieval.

11.2 Uses

- `Svc.Session` – for persisting session data to backend storage.
- `UI.Canvas` – for retrieving current canvas state.

11.3 Syntax

11.3.1 Exported Constants

- `MAX_SESSION_NAME_LENGTH = 100`

11.3.2 Exported Access Programs

Table 8: UI.SaveSession - Exported Access Programs

Name	In	Out	Exceptions
`renderSaveDialog`	`none`	`none`	`UIRenderError`
`saveCurrentSession`	`name: String`	`SessionID`	`ValidationError, NetworkError`
`validateSessionName`	`name: String`	`bool`	`ValidationError`

11.4 Semantics

11.4.1 State Variables

- `sessionName: String`
- `saveInProgress: bool`
- `lastSavedID: SessionID or null`

11.4.2 Environment Variables

- Browser DOM for dialog rendering.

11.4.3 Assumptions

- User is authenticated and has valid session context.
- Canvas state is available and valid at time of save.

11.4.4 Access Routine Semantics

`renderSaveDialog()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: displays modal dialog with input field for session name and save button.
- output: none.
- exception: `UIRenderError` if dialog cannot be rendered.

`saveCurrentSession(name)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: validates session name; captures current canvas state from `UI.Canvas`; sends save request to `Svc.Session`; updates `lastSavedID` on success.
- output: `SessionID` assigned by backend.
- exception: `ValidationError` if name invalid; `NetworkError` if save fails.

`validateSessionName(name)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `true` if name meets length and character requirements.
- exception: `ValidationError` if name invalid.

11.4.5 Local Functions

- `showSuccessMessage(id)` – displays confirmation with session ID.
- `closeDialog()` – closes save dialog and clears state.

12 MIS of UI.LoadSession

12.1 Module

UI.LoadSession (Session Load Interface)

This module provides the user interface for browsing, selecting, and loading previously saved workspace sessions. It displays session metadata and allows users to restore their reconstruction work.

12.2 Uses

- `Svc.Session` – for retrieving available sessions and loading session data.
- `UI.Canvas` – for applying loaded state to the canvas.

12.3 Syntax

12.3.1 Exported Constants

- `SESSIONS_PAGE_SIZE = 10`

12.3.2 Exported Access Programs

Table 9: UI.LoadSession - Exported Access Programs

Name	In	Out	Exceptions
`renderLoadDialog`	`none`	`none`	`UIRenderError`
`fetchSessionList`	`none`	`SessionList`	`NetworkError`
`loadSession`	`id: SessionID`	`none`	`NotFoundError, NetworkError`
`deleteSession`	`id: SessionID`	`bool`	`NetworkError`

12.4 Semantics

12.4.1 State Variables

- `availableSessions: SessionList`
- `selectedSessionID: SessionID or null`
- `loadInProgress: bool`

12.4.2 Environment Variables

- Browser DOM for dialog rendering and session list display.

12.4.3 Assumptions

- User is authenticated and has valid session context.
- Session data retrieved from backend is valid and compatible with current system version.

12.4.4 Access Routine Semantics

`renderLoadDialog()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.

- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: displays modal dialog with list of saved sessions and load button.
- output: none.
- exception: `UIRenderError` if dialog cannot be rendered.

`fetchSessionList()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: requests list of available sessions from `Svc.Session`; updates `availableSessions`.
- output: `SessionList` containing session metadata (ID, name, date, preview).
- exception: `NetworkError` if request fails.

`loadSession(id)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: retrieves session data from `Svc.Session`; applies state to `UI.Canvas`; closes load dialog.
- output: none.
- exception: `NotFoundError` if session does not exist; `NetworkError` if retrieval fails.

`deleteSession(id)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: sends delete request to `Svc.Session`; removes session from `availableSessions` on success.
- output: `true` if deletion successful.
- exception: `NetworkError` if deletion fails.

12.4.5 Local Functions

- `renderSessionList()` – updates DOM with session list.
- `confirmDelete(id)` – shows confirmation dialog before deletion.
- `closeDialog()` – closes load dialog and clears state.

13 MIS of Svc.API

13.1 Module

Svc.API (Backend API Gateway)

This module exposes the backend functionality of the Sanskrit Manuscript Fragment Reconstruction Platform as a REST-style service layer. It routes incoming HTTP requests from the UI to the corresponding internal services, enforces basic request validation, and standardizes API responses.

13.2 Uses

- `Svc.AuthZ` – for authentication and authorization checks.
- `Svc.Session` – for resolving current user/session context.
- `Svc.ProcOrch` – for preprocessing and normalization requests.
- `Svc.MatchOrch` – for fragment match discovery.
- `Svc.Search` – for database search and exploration.
- `Data.FragmentStore`, `Data.User`, `Data.Project`.

13.3 Syntax

13.3.1 Exported Constants

- `API_VERSION = "v1"`
- `MAX_REQUEST_SIZE` – logical upper bound on payload size (aligned with SRS).

13.3.2 Exported Access Programs

Table 10: Svc.API - Exported Access Programs

Name	In	Out	Exceptions
`startApplication`	`none`	`none`	`ServiceError`
`handleAuthRequest`	`req: HttpRequest`	`HttpResponse`	`AuthError`
`handleUploadRequest`	`req: HttpRequest`	`HttpResponse`	`ValidationError, ServiceError`
`handleMatchRequest`	`req: HttpRequest`	`HttpResponse`	`ServiceError`
`handleSearchRequest`	`req: HttpRequest`	`HttpResponse`	`ValidationError, ServiceError`
`handleSessionRequest`	`req: HttpRequest`	`HttpResponse`	`ServiceError`

13.4 Semantics

13.4.1 State Variables

- `appInitialized: bool` – true once handlers and service bindings are registered.
- `routeTable: Map[Endpoint, Handler]` – mapping between URL paths and internal handlers.

13.4.2 Environment Variables

- HTTP server environment providing incoming requests and outgoing responses.

13.4.3 Assumptions

- All requests include sufficient information to resolve user/session context when required.
- Lower-level services enforce their own invariants in addition to basic checks at this layer.

13.4.4 Access Routine Semantics

`startApplication()`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.

- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: initializes the route table, binds handlers to the application service boundary, and marks `appInitialized` as `true`.
- output: none.
- exception: `ServiceError` if the service boundary cannot be initialized.

`handleAuthRequest(req)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: forwards credentials/token to `Svc.AuthZ`; logs outcome.
- output: `HttpResponse` with status code and (if successful) session token or user info.
- exception: `AuthError` if authentication fails or token invalid.

`handleUploadRequest(req)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: validates payload size and type; forwards work to `Svc.ProcOrch` for ingestion.
- output: `HttpResponse` including created fragment IDs or error details.
- exception: `ValidationError` if request violates constraints; `ServiceError` if downstream failure occurs.

`handleMatchRequest(req)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: parses fragment identifier and forwards to `Svc.MatchOrch`.

- output: `HttpResponse` containing ranked match suggestions.
- exception: `ServiceError` if orchestrator fails or times out.

`handleSearchRequest(req)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: validates search query parameters; forwards to `Svc.Search`.
- output: `HttpResponse` containing search results page.
- exception: `ValidationError` for malformed queries; `ServiceError` on backend failures.

`handleSessionRequest(req)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: routes session save/load actions to `Svc.Session`.
- output: `HttpResponse` with session payload or confirmation.
- exception: `ServiceError` if the session operation fails.

13.4.5 Local Functions

- `initializeRouteTable()` – registers the handlers used by the application entrypoint.
- `parseJSONBody(req)` – extracts and validates JSON payload.
- `buildErrorResponse(code, msg)` – standardizes error responses.

14 MIS of Svc.AuthZ

14.1 Module

Svc.AuthZ (Authentication and Authorization Service)

This module validates user credentials, issues and verifies session tokens, and enforces access control for protected operations.

14.2 Uses

- `Data.User` – for user credentials and role lookup.
- Cryptographic utilities (hashing, token signing).

14.3 Syntax

14.3.1 Exported Constants

- `TOKEN_TTL_MINUTES` – session token lifetime.
- `MAX_FAILED_ATTEMPTS` – lockout threshold.

14.3.2 Exported Access Programs

Table 11: Svc.AuthZ - Exported Access Programs

Name	In	Out	Exceptions
`authenticate`	`creds: UserCredentials`	`AuthResult`	`AuthError`
`validateToken`	`token: String`	`UserContext`	`AuthError`
`checkAccess`	`ctx: UserContext, action: String`	`bool`	`AuthError`

14.4 Semantics

14.4.1 State Variables

- `failedAttempts: Map[UserID, int]`

14.4.2 Environment Variables

- Secure storage for password hashes and secret keys.

14.4.3 Assumptions

- Passwords are stored as salted hashes only.
- Time and randomness sources are trustworthy for token generation.

14.4.4 Access Routine Semantics

`authenticate(creds)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: verifies credentials against stored hash; updates `failedAttempts`.
- output: `AuthResult` containing success flag and token on success.
- exception: `AuthError` if user locked out or credentials invalid.

`validateToken(token)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `UserContext` if token valid and not expired.
- exception: `AuthError` if token invalid or expired.

`checkAccess(ctx, action)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `true` if user role in `ctx` is allowed to perform `action`.
- exception: `AuthError` if context invalid.

14.4.5 Local Functions

- `hashPassword(pw)` – hashes password for comparison.
- `generateToken(user)` – constructs signed token.

15 MIS of Svc.Session

15.1 Module

Svc.Session (Session and Workspace Management Service)

This module manages user sessions, saved canvas layouts, and ongoing reconstruction projects. It provides persistent storage and retrieval of user workspace state.

15.2 Uses

- `Data.Project` – for project-level data and references.
- `Data.User` – for associating sessions with users.

15.3 Syntax

15.3.1 Exported Constants

- `MAX_SAVED_LAYOUTS_PER_USER`

15.3.2 Exported Access Programs

Table 12: Svc.Session - Exported Access Programs

Name	In	Out	Exceptions
`saveCanvasState`	`ctx: UserContext, state: CanvasState`	`LayoutID`	`SessionError`
`loadCanvasState`	`ctx: UserContext, id: LayoutID`	`CanvasState`	`NotFoundError, SessionError`
`listLayouts`	`ctx: UserContext`	`Sequence[LayoutSummary]`	`SessionError`

15.4 Semantics

15.4.1 State Variables

- `layoutStore: Map[LayoutID, (UserID, CanvasState)]`

15.4.2 Environment Variables

- Persistent database for layouts and project metadata.

15.4.3 Assumptions

- `UserContext` is valid and has been authenticated by `Svc.AuthZ`.

15.4.4 Access Routine Semantics

`saveCanvasState(ctx, state)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: creates or updates an entry in `layoutStore` for the user.
- output: generated `LayoutID`.
- exception: `SessionError` if write fails or limit exceeded.

`loadCanvasState(ctx, id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `CanvasState` associated with user and layout ID.
- exception: `NotFoundError` if layout does not exist or not owned by user; `SessionError` on read failure.

`listLayouts(ctx)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: summaries of layouts owned by the user.
- exception: `SessionError` if retrieval fails.

15.4.5 Local Functions

- `generateLayoutID()` – creates unique identifier for layouts.

16 MIS of Svc.ProcOrch

16.1 Module

Svc.ProcOrch (Preprocessing and Normalization Orchestrator)

This module coordinates image preprocessing workflows such as orientation correction, denoising, and resolution standardization before downstream analysis.

16.2 Uses

- `HW.ImageStorage` – for reading and writing image files.
- `ML.EdgeMatch`, `ML.Damage` – where preprocessing parameters are shared.
- External image processing library (for example, OpenCV).

16.3 Syntax

16.3.1 Exported Constants

- `TARGET_RESOLUTION` – canonical resolution for normalized images.
- `MAX_ORIENTATION_ERROR` – tolerated post-correction skew.

16.3.2 Exported Access Programs

Table 13: Svc.ProcOrch - Exported Access Programs

Name	In	Out	Exceptions
`enqueuePreprocessing`	`id: ImageID`	`JobID`	`PreprocError`
`runPreprocessing`	`job: JobID`	`PreprocResult`	`PreprocError`

16.4 Semantics

16.4.1 State Variables

- `jobQueue: Sequence[JobID]`

16.4.2 Environment Variables

- Execution environment for batch or async jobs.

16.4.3 Assumptions

- Input image already stored by `HW.ImageStorage`.

16.4.4 Access Routine Semantics

`enqueuePreprocessing(id)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: creates a preprocessing job for `id` and appends it to `jobQueue`.
- output: `JobID` for tracking.
- exception: `PreprocError` if job cannot be created.

`runPreprocessing(job)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: retrieves image by `ImageID`, applies normalization steps, and writes results back to storage.
- output: `PreprocResult` containing normalized image reference and metrics (for example, estimated orientation error).
- exception: `PreprocError` if any step fails.

16.4.5 Local Functions

- `correctOrientation(img)` – aligns text horizontally.
- `normalizeResolution(img)` – resamples to `TARGET_RESOLUTION`.

17 MIS of Svc.MatchOrch

17.1 Module

Svc.MatchOrch (Fragment Matching Orchestrator)

This module coordinates AI models for fragment matching, aggregates model outputs, and produces ranked match suggestions for a given fragment.

17.2 Uses

- `ML.EdgeMatch`, `ML.Damage`.
- `Data.FragmentStore` – for retrieving fragment metadata.

17.3 Syntax

17.3.1 Exported Constants

- `MAX_TOP_MATCHES = 10`
- `MATCH_CONFIDENCE_THRESHOLD = 0.6`

17.3.2 Exported Access Programs

Table 14: Svc.MatchOrch - Exported Access Programs

Name	In	Out	Exceptions
`generateMatches`	`frag: FragmentID`	`MatchResultList`	`MatchError`
`aggregateScores`	`scores: Sequence[ModelScore]`	`float`	`AggregationError`

17.4 Semantics

17.4.1 State Variables

- `recentResults: Map[FragmentID, MatchResultList]`

17.4.2 Environment Variables

- ML model serving endpoints.

17.4.3 Assumptions

- Each model returns normalized scores in $[0, 1]$.

17.4.4 Access Routine Semantics

`generateMatches(frag)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.

- transition: queries all relevant ML models; aggregates and sorts candidate matches; updates `recentResults`.
- output: `MatchResultList` of top candidates above threshold.
- exception: `MatchError` if models unavailable or response invalid.

`aggregateScores(scores)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: combined score (for example, weighted mean) over model scores.
- exception: `AggregationError` if sequence empty or weights invalid.

17.4.5 Local Functions

- `filterByThreshold(results)` – drops candidates below `MATCH_CONFIDENCE_THRESHOLD`.
- `sortByScore(results)` – orders candidates descending by final score.

18 MIS of Svc.Search

18.1 Module

Svc.Search (Search and Exploration Service)

This module executes queries over stored fragments and metadata, providing paginated and filterable search results to the UI.

18.2 Uses

- `Data.FragmentStore`, `Data.Catalog`.

18.3 Syntax

18.3.1 Exported Constants

- `DEFAULT_PAGE_SIZE = 25`

18.3.2 Exported Access Programs

Table 15: Svc.Search - Exported Access Programs

Name	In	Out	Exceptions
`executeQuery`	`q: SearchQuery, page: int`	`SearchResultPage`	`SearchError`
`exportResults`	`q: SearchQuery, format: ExportFormat`	`ExportPayload`	`SearchError, ExportError`

18.4 Semantics

18.4.1 State Variables

- none

18.4.2 Environment Variables

- Database engine for indexing and querying fragment records.

18.4.3 Assumptions

- All searchable fields are indexed appropriately for performance.

18.4.4 Access Routine Semantics

`executeQuery(q, page)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `SearchResultPage` containing fragment summaries for the requested page.
- exception: `SearchError` if query invalid or backend failure occurs.

`exportResults(q, format)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.

- output: `ExportPayload` that can be streamed to client as CSV or PDF.
- exception: `SearchError` on query failure; `ExportError` if export generation fails.

18.4.5 Local Functions

- `buildQueryPlan(q)` – compiles filter and sort options into database query.
- `formatExport(results, format)` – converts results into specified output format.

19 MIS of Data.FragmentStore

19.1 Module

Data.FragmentStore (Fragment Metadata Store)

This module is responsible for storing and retrieving logical fragment entities. It maintains metadata such as fragment IDs, associated image IDs, script labels, line counts, and status flags, and acts as the core record of fragments used throughout the system.

19.2 Uses

- `HW.ImageStorage` – via stored `ImageID` references.
- `Data.Catalog` – for linking to external catalogue identifiers when available.

19.3 Syntax

19.3.1 Exported Constants

- `MAX_LABEL_LENGTH`

19.3.2 Exported Access Programs

Table 16: Data.FragmentStore - Exported Access Programs

Name	In	Out	Exceptions
`createFragment`	`rec: FragmentRecord`	`FragmentID`	`FragmentError`
`getFragment`	`id: FragmentID`	`FragmentRecord`	`NotFoundError`
`updateFragment`	`id: FragmentID, rec: FragmentRecord`	`bool`	`FragmentError, NotFoundError`
`deleteFragment`	`id: FragmentID`	`bool`	`FragmentError, NotFoundError`

19.4 Semantics

19.4.1 State Variables

- `fragmentTable: Map[FragmentID, FragmentRecord]`

19.4.2 Environment Variables

- Relational database or equivalent persistent storage.

19.4.3 Assumptions

- Fragment IDs are globally unique and stable over time.

19.4.4 Access Routine Semantics

`createFragment(rec)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: inserts a new record into `fragmentTable`.
- output: new `FragmentID`.
- exception: `FragmentError` if insert fails or constraints violated.

`getFragment(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: record associated with `id`.
- exception: `NotFoundError` if no record exists.

`updateFragment(id, rec)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.

- transition: replaces existing record for `id` with `rec`.
- output: `true` if update successful.
- exception: `NotFoundError` if `id` unknown; `FragmentError` on write failure.

`deleteFragment(id)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: removes record associated with `id`.
- output: `true` if deletion successful.
- exception: `NotFoundError` if `id` unknown; `FragmentError` on delete failure.

19.4.5 Local Functions

- `validateFragment(rec)` – checks required fields and constraints.

20 MIS of Data.Catalog

20.1 Module

Data.Catalog (External Catalogue Link Store)

This module maintains mappings between internal fragments and external catalogue identifiers from libraries, collections, or previous editions.

20.2 Uses

- `Data.FragmentStore` – for core fragment records.

20.3 Syntax

20.3.1 Exported Constants

- `MAX_CATALOG_ID_LENGTH`

20.3.2 Exported Access Programs

Table 17: Data.Catalog - Exported Access Programs

Name	In	Out	Exceptions
`linkFragmentToCatalog`	`frag: FragmentID, catID: String`	`bool`	`CatalogError`
`getCatalogIDs`	`frag: FragmentID`	`Sequence[String]`	`CatalogError`
`findByCatalogID`	`catID: String`	`Sequence[FragmentID]`	`CatalogError`

20.4 Semantics

20.4.1 State Variables

- `catalogMap: Map[FragmentID, Sequence[String]]`

20.4.2 Environment Variables

- Same underlying database as the fragment store or a logically related schema.

20.4.3 Assumptions

- External catalogue identifiers are opaque strings; uniqueness is handled externally.

20.4.4 Access Routine Semantics

`linkFragmentToCatalog(frag, catID)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: appends `catID` to the list of identifiers for `frag`.
- output: `true` on success.
- exception: `CatalogError` on write failure.

`getCatalogIDs(frag)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.

- output: all catalogue IDs associated with `frag`.
- exception: `CatalogError` if retrieval fails.

`findByCatalogID(catID)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: all `FragmentID` values linked to `catID`.
- exception: `CatalogError` if query fails.

20.4.5 Local Functions

- `normalizeCatalogID(catID)` – optional normalization (trim, case).

21 MIS of Data.User

21.1 Module

Data.User (User Account Store)

This module stores user accounts, credentials (hashed), roles, and profile information required for authentication and access control.

21.2 Uses

- Cryptographic utilities for password hashing.

21.3 Syntax

21.3.1 Exported Constants

- `MAX_USERNAME_LENGTH`
- `MAX_EMAIL_LENGTH`

21.3.2 Exported Access Programs

Table 18: Data.User - Exported Access Programs

Name	In	Out	Exceptions
`createUser`	`rec: UserRecord`	`UserID`	`UserError`
`getUserByID`	`id: UserID`	`UserRecord`	`NotFoundError`
`getUserByEmail`	`email: String`	`UserRecord`	`NotFoundError`
`updateUser`	`id: UserID, rec: UserRecord`	`bool`	`UserError, NotFoundError`

21.4 Semantics

21.4.1 State Variables

- `userTable: Map[UserID, UserRecord]`

21.4.2 Environment Variables

- Persistent user database with appropriate access controls.

21.4.3 Assumptions

- Email addresses are unique per user.

21.4.4 Access Routine Semantics

`createUser(rec)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: inserts new user record; hashes password fields before storage.
- output: generated `UserID`.
- exception: `UserError` if constraints violated or write fails.

`getUserByID(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.

- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: user record with `id`.
- exception: `NotFoundError` if no such user.

`getUserByEmail(email)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: user record whose email matches `email`.
- exception: `NotFoundError` if not found.

`updateUser(id, rec)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: updates record for `id`.
- output: `true` on success.
- exception: `NotFoundError` if `id` unknown; `UserError` on failure.

21.4.5 Local Functions

- `validateUser(rec)` – checks email format, password policy etc.

22 MIS of Data.Project

22.1 Module

Data.Project (Project and Reconstruction Session Store)

This module records reconstruction projects, including project metadata, associated fragments, and saved reconstruction sessions.

22.2 Uses

- `Data.User` – to associate projects with owners or collaborators.
- `Data.FragmentStore`.

22.3 Syntax

22.3.1 Exported Constants

- `MAX_PROJECT_NAME_LENGTH`

22.3.2 Exported Access Programs

Table 19: Data.Project - Exported Access Programs

Name	In	Out	Exceptions
`createProject`	`rec: ProjectRecord`	`ProjectID`	`ProjectError`
`getProject`	`id: ProjectID`	`ProjectRecord`	`NotFoundError`
`updateProject`	`id: ProjectID, rec: ProjectRecord`	`bool`	`ProjectError, NotFoundError`
`listProjectsForUser`	`user: UserID`	`Sequence[ProjectSummary]`	`ProjectError`

22.4 Semantics

22.4.1 State Variables

- `projectTable: Map[ProjectID, ProjectRecord]`

22.4.2 Environment Variables

- Persistent project database.

22.4.3 Assumptions

- Each project belongs to at least one user.

22.4.4 Access Routine Semantics

`createProject(rec)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.

- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: inserts new row in `projectTable`.
- output: created `ProjectID`.
- exception: `ProjectError` on constraint violation or failure.

`getProject(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: project record matching `id`.
- exception: `NotFoundError` if none exists.

`updateProject(id, rec)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.
- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: updates stored record.
- output: `true` on success.
- exception: `NotFoundError` or `ProjectError`.

`listProjectsForUser(user)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: summaries of all projects owned by or shared with `user`.
- exception: `ProjectError` if query fails.

22.4.5 Local Functions

- `validateProject(rec)` – checks required fields and name length.

23 MIS of ML.EdgeMatch

23.1 Module

ML.EdgeMatch (Edge Pattern Matching Model)

This module provides model-based similarity estimates based on fragment edge contours and shapes.

23.2 Uses

- `HW.ImageStorage` – to retrieve normalized images.
- `Util.Error` – for constructing `ModelError` exceptions.

23.3 Syntax

23.3.1 Exported Constants

- `EDGE_SIGNATURE_DIM`

23.3.2 Exported Access Programs

Table 20: ML.EdgeMatch - Exported Access Programs

Name	In	Out	Exceptions
`computeEdgeSignature`	`id: ImageID`	`EdgeSignature`	`ModelError`
`scoreEdgeSimilarity`	`a: EdgeSignature, b: EdgeSignature`	`float`	`ModelError`

23.4 Semantics

23.4.1 State Variables

- Optional cached model weights and configuration.

23.4.2 Environment Variables

- Model runtime (for example, PyTorch or TensorFlow backend).

23.4.3 Assumptions

- Input images have been preprocessed and normalized by `Svc.ProcOrch`.

23.4.4 Access Routine Semantics

`computeEdgeSignature(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: fixed-length vector representing edge features for the image.
- exception: `ModelError` if image cannot be processed.

`scoreEdgeSimilarity(a, b)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: similarity score in $[0, 1]$ reflecting how well edges match.
- exception: `ModelError` if signatures invalid or incompatible.

23.4.5 Local Functions

- `extractContours(img)` – local function to compute edge features.

24 MIS of ML.ScriptClass

24.1 Module

ML.ScriptClass (Script Classification Model)

This module classifies the script type of a fragment (for example, Gupta Brāhmī, Tibetan Uchen, Chinese) and returns confidence scores.

24.2 Uses

- `HW.ImageStorage` – for retrieving fragment images.
- `Util.Error` – for constructing `ModelError` exceptions.

24.3 Syntax

24.3.1 Exported Constants

- `SUPPORTED_SCRIPTS: Sequence[String]`

24.3.2 Exported Access Programs

Table 21: ML.ScriptClass - Exported Access Programs

Name	In	Out	Exceptions
`classifyScript`	`id: ImageID`	`ScriptPrediction`	`ModelError`

24.4 Semantics

24.4.1 State Variables

- Model weights for script classification.

24.4.2 Environment Variables

- Model runtime and hardware accelerators (if available).

24.4.3 Assumptions

- Input image includes at least some legible text.

24.4.4 Access Routine Semantics

`classifyScript(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `ScriptPrediction` containing predicted script label and confidence.
- exception: `ModelError` if classification fails or confidence undefined.

24.4.5 Local Functions

- `preprocessForScript(img)` – applies script-specific preprocessing pipeline.

25 MIS of ML.TextSim

25.1 Module

ML.TextSim (Textual Content Similarity Model)

This module computes semantic similarity between text segments derived from transcribed text or manual annotations.

25.2 Uses

- `Data.FragmentStore` – for accessing fragment text content.
- `Util.Error` – for constructing `ModelError` exceptions.

25.3 Syntax

25.3.1 Exported Constants

- `EMBEDDING_DIM`

25.3.2 Exported Access Programs

Table 22: ML.TextSim - Exported Access Programs

Name	In	Out	Exceptions
`embedText`	`text: String`	`TextEmbedding`	`ModelError`
`scoreTextSimilarity`	`a: TextEmbedding, b: TextEmbedding`	`float`	`ModelError`

25.4 Semantics

25.4.1 State Variables

- Embedding model weights.

25.4.2 Environment Variables

- Model runtime.

25.4.3 Assumptions

- Input text is normalized (script, encoding).

25.4.4 Access Routine Semantics

`embedText(text)`:

- output: fixed-length embedding vector.
- exception: `ModelError` if model fails.

`scoreTextSimilarity(a, b)`:

- output: similarity score in $[0, 1]$.
- exception: `ModelError` if inputs invalid.

25.4.5 Local Functions

- `cosineSimilarity(a, b)` – core similarity computation.

26 MIS of ML.Segmentation

26.1 Module

ML.Segmentation (Fragment Segmentation Model)

This module performs image segmentation to identify and isolate individual fragment regions from manuscript images. It detects fragment boundaries and separates multiple fragments that may appear in a single scanned image.

26.2 Uses

- `HW.ImageStorage` – for retrieving images to segment.
- `Util.Error` – for constructing `ModelError` exceptions.

26.3 Syntax

26.3.1 Exported Constants

- `MIN_FRAGMENT_SIZE` – minimum area threshold for valid fragments.
- `SEGMENTATION_CONFIDENCE_THRESHOLD = 0.7`

26.3.2 Exported Access Programs

Table 23: ML.Segmentation - Exported Access Programs

Name	In	Out	Exceptions
segmentImage	id: ImageID	SegmentationResult	ModelError
extractFragments	id: ImageID, masks: SegmentMaskList	Sequence[ImageID]	ModelError

26.4 Semantics

26.4.1 State Variables

- Segmentation model weights and configuration.

26.4.2 Environment Variables

- Model runtime environment for deep learning inference.

26.4.3 Assumptions

- Input images have been preprocessed (orientation correction, resolution normalization).
- Fragments in the image have sufficient contrast with the background.

26.4.4 Access Routine Semantics

`segmentImage(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `SegmentationResult` containing binary masks and bounding boxes for detected fragments.
- exception: `ModelError` if segmentation fails or image cannot be processed.

`extractFragments(id, masks)`:

- pre: inputs satisfy the module assumptions, and any required identifiers, state, or external resources referenced by this routine are valid and available.

- post: if no exception is raised, the documented state transition is completed and the specified output is produced.
- transition: creates separate image files for each detected fragment region; stores them via `HW.ImageStorage`.
- output: sequence of new `ImageID` values for extracted fragments.
- exception: `ModelError` if extraction or storage fails.

26.4.5 Local Functions

- `applyMorphology(mask)` – cleans segmentation masks using morphological operations.
- `filterSmallRegions(masks)` – removes regions below `MIN_FRAGMENT_SIZE`.

27 MIS of ML.CircleRecognition

27.1 Module

ML.CircleRecognition (Circle Detection and Recognition Model)

This module detects circular marks, holes, or decorative elements in manuscript fragments. These features can serve as additional matching criteria when aligning fragments.

27.2 Uses

- `HW.ImageStorage` – for accessing fragment images.
- `Util.Error` – for constructing `ModelError` exceptions.

27.3 Syntax

27.3.1 Exported Constants

- `MIN_CIRCLE_RADIUS` – minimum detectable circle size in pixels.
- `MAX_CIRCLE_RADIUS` – maximum detectable circle size in pixels.

27.3.2 Exported Access Programs

Table 24: ML.CircleRecognition - Exported Access Programs

Name	In	Out	Exceptions
`detectCircles`	`id: ImageID`	`CircleList`	`ModelError`
`matchCirclePatterns`	`a: CircleList, b: CircleList`	`float`	`ModelError`

27.4 Semantics

27.4.1 State Variables

- Detection parameters and thresholds.

27.4.2 Environment Variables

- Image processing library (e.g., OpenCV).

27.4.3 Assumptions

- Circles are sufficiently distinct from background texture.
- Images are preprocessed and normalized.

27.4.4 Access Routine Semantics

`detectCircles(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `CircleList` containing center coordinates, radii, and confidence scores for detected circles.
- exception: `ModelError` if detection fails.

`matchCirclePatterns(a, b)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.

- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: similarity score in $[0, 1]$ based on spatial arrangement and properties of circles.
- exception: `ModelError` if matching algorithm fails.

27.4.5 Local Functions

- `computeSpatialHash(circles)` – creates geometric hash for circle arrangement.
- `filterOverlappingCircles(circles)` – removes duplicate detections.

28 MIS of ML.LineCount

28.1 Module

ML.LineCount (Text Line Counting Model)

This module analyzes manuscript fragments to count the number of text lines present. Line count serves as a useful filtering and matching criterion during fragment search and reconstruction.

28.2 Uses

- `HW.ImageStorage` – for accessing fragment images.
- `Util.Error` – for constructing `ModelError` exceptions.

28.3 Syntax

28.3.1 Exported Constants

- `MIN_LINE_SPACING` – minimum expected spacing between lines in pixels.

28.3.2 Exported Access Programs

Table 25: ML.LineCount - Exported Access Programs

Name	In	Out	Exceptions
`countLines`	`id: ImageID`	`LineCountResult`	`ModelError`
`detectLineRegions`	`id: ImageID`	`Sequence[BoundingBox]`	`ModelError`

28.4 Semantics

28.4.1 State Variables

- Line detection model parameters.

28.4.2 Environment Variables

- Image processing and model inference environment.

28.4.3 Assumptions

- Text lines are approximately horizontal after preprocessing.
- Sufficient contrast exists between text and background.

28.4.4 Access Routine Semantics

`countLines(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `LineCountResult` containing the number of detected lines and confidence score.
- exception: `ModelError` if counting fails or image quality insufficient.

`detectLineRegions(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: sequence of `BoundingBox` objects defining the spatial extent of each text line.
- exception: `ModelError` if line detection fails.

28.4.5 Local Functions

- `projectHorizontally(img)` – creates horizontal projection histogram for line detection.
- `findLineBoundaries(projection)` – identifies line separation points in projection.

29 MIS of ML.EdgeClassification

29.1 Module

ML.EdgeClassification (Edge Type Classification Model)

This module classifies fragment edges into categories such as torn, cut, original (manuscript boundary), or burned. Edge type classification helps improve matching accuracy by ensuring compatible edge pairs are compared.

29.2 Uses

- `HW.ImageStorage` – for accessing fragment images.
- `ML.EdgeMatch` – edge signatures may be shared for classification.
- `Util.Error` – for constructing `ModelError` exceptions.

29.3 Syntax

29.3.1 Exported Constants

- `EDGE_TYPES =`
  `torn, cut, original, burned, unknown`
- `MIN_CLASSIFICATION_CONFIDENCE = 0.6`

29.3.2 Exported Access Programs

Table 26: ML.EdgeClassification - Exported Access Programs

Name	In	Out	Exceptions
`classifyEdges`	`id: ImageID`	`EdgeClassificationResult`	`ModelError`
`getEdgeType`	`id: ImageID, edge: EdgeRegion`	`EdgeTypePrediction`	`ModelError`

29.4 Semantics

29.4.1 State Variables

- Edge classification model weights.

29.4.2 Environment Variables

- Model runtime for deep learning inference.

29.4.3 Assumptions

- Fragment edges are clearly visible in preprocessed images.
- Training data covers representative examples of each edge type.

29.4.4 Access Routine Semantics

`classifyEdges(id)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `EdgeClassificationResult` containing edge type predictions for all detected edges with confidence scores.
- exception: `ModelError` if classification fails.

`getEdgeType(id, edge)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: `EdgeTypePrediction` for a specific edge region containing predicted type and confidence.
- exception: `ModelError` if edge region invalid or classification fails.

29.4.5 Local Functions

- `extractEdgeFeatures(img, region)` – computes texture and shape features for edge region.
- `normalizeConfidences(predictions)` – adjusts confidence scores to sum to 1.0.

30 MIS of Util.Error

30.1 Module

Util.Error (Error Type and Exception Utilities)

This module defines standard error types and helpers for constructing and handling errors in a consistent way across modules.

30.2 Uses

- None at the architectural level; referenced by many other modules.

30.3 Syntax

30.3.1 Exported Constants

- Standard error codes or identifiers (for example, `ERR_NOT_FOUND`, `ERR_VALIDATION`).

30.3.2 Exported Access Programs

Table 27: Util.Error - Exported Access Programs

Name	In	Out	Exceptions
`buildError`	`code: String, msg: String`	`ErrorObject`	`None`
`toClientMessage`	`err: ErrorObject`	`String`	`None`

30.4 Semantics

30.4.1 State Variables

- None.

30.4.2 Environment Variables

- None.

30.4.3 Assumptions

- All modules reference the same canonical error types.

30.4.4 Access Routine Semantics

`buildError(code, msg)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: structured error object capturing code and message.
- exception: none.

`toClientMessage(err)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: safe message for display at UI or API boundary.
- exception: none.

31 MIS of Util.Logging

31.1 Module

Util.Logging (Logging Utility)
This module provides structured logging for events, errors, and performance metrics.

31.2 Uses

- Underlying logging framework or standard output.

31.3 Syntax

31.3.1 Exported Constants

- Log levels such as `INFO`, `WARN`, `ERROR`.

31.3.2 Exported Access Programs

Table 28: Util.Logging - Exported Access Programs

Name	In	Out	Exceptions
`logInfo`	`msg: String`	`none`	`None`
`logWarn`	`msg: String`	`none`	`None`
`logError`	`msg: String, err: ErrorObject`	`none`	`None`

31.4 Semantics

31.4.1 State Variables

- Optional runtime configuration (log level, sinks).

31.4.2 Environment Variables

- File system, console, or external logging service.

31.4.3 Assumptions

- Logging operations are non-fatal and do not throw.

31.4.4 Access Routine Semantics

`logInfo(msg)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- transition: writes informational log entry.
- output: none.
- exception: none.

`logWarn(msg)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- transition: writes warning log entry.
- output: none.
- exception: none.

`logError(msg, err)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- transition: writes error entry including details from `err`.
- output: none.
- exception: none.

32 MIS of Util.Config

32.1 Module

Util.Config (Configuration Management)

This module provides read-only access to configuration values such as database connection strings, model paths, and feature flags.

32.2 Uses

- Environment variables or configuration files.

32.3 Syntax

32.3.1 Exported Constants

- None fixed; keys are used to look up values.

32.3.2 Exported Access Programs

Table 29: Util.Config - Exported Access Programs

Name	In	Out	Exceptions
`getValue`	`key: String`	`String`	`ConfigError`
`getBool`	`key: String`	`bool`	`ConfigError`

32.4 Semantics

32.4.1 State Variables

- `configCache: Map[String, String]`

32.4.2 Environment Variables

- Config file or environment variable store.

32.4.3 Assumptions

- Configuration is loaded at startup and remains static during runtime.

32.4.4 Access Routine Semantics

`getValue(key)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: configuration value associated with `key`.
- exception: `ConfigError` if key missing.

`getBool(key)`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: parsed boolean value.
- exception: `ConfigError` if key missing or value not parseable.

33 MIS of Util.TestStub

33.1 Module

Util.TestStub (Testing Stubs and Fakes)

This module provides stub implementations of selected services and ML components for use in automated tests.

33.2 Uses

- Core service and ML interfaces for type compatibility.

33.3 Syntax

33.3.1 Exported Constants

- None.

33.3.2 Exported Access Programs

Table 30: Util.TestStub - Exported Access Programs

Name	In	Out	Exceptions
`getFixedOCRStub`	`none`	`ML.OCR-like object`	`None`
`getFixedMatchStub`	`none`	`Svc.MatchOrch-like object`	`None`

33.4 Semantics

33.4.1 State Variables

- Predefined responses used for deterministic tests.

33.4.2 Environment Variables

- None.

33.4.3 Assumptions

- Stubs are only used in test environments.

33.4.4 Access Routine Semantics

`getFixedOCRStub()`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: object implementing `runOCR` that returns predictable results for known inputs.
- exception: none.

`getFixedMatchStub()`:

- pre: inputs satisfy the module assumptions, and any arguments or referenced identifiers used by this routine are valid.
- post: if no exception is raised, the specified output is produced and module state is unchanged unless stated otherwise below.
- output: object implementing `generateMatches` with fixed candidate lists.
- exception: none.

33.4.5 Local Functions

- `loadStubFixtures()` – loads predefined responses from static files.

References

Team Sanskrit Ciphers. Software requirements specification (srs). https://github.com/DylanG5/sanskrit-ciphers-requirements/blob/9938018b682709551d0a6248a0a9f6e5794112ee/index.pdf, 2025. Specifies functional and non-functional requirements for the system.

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

1. What went well while writing this deliverable?

   One aspect that went particularly well was identifying the appropriate notation and determining which modules needed to be included in the design. Our team communicated effectively throughout this process, which helped us maintain a shared understanding of the system and its structure. Because we already had a strong grasp of the project vision, it was relatively easy to map our ideas into a formal design representation. As a result, we were able to present a clear and coherent picture of the system architecture that reflects both the requirements and the intended functionality.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   The main challenge we encountered was thinking through the lower-level implementation details. At this stage, we have not yet progressed far into the actual coding, so some of the design decisions, especially those related to specific algorithms, data structures, and system interactions required educated assumptions rather than concrete experience. We addressed this by focusing on higher-level abstractions and deferring detailed implementation decisions until we gain more clarity during development. This allowed us to complete the deliverable while still leaving room for necessary adjustments later.

3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?

   Many of our design decisions, especially those involving the user interface, were directly influenced by conversations with our supervisor, Dr. Shayne. Since he does not come from a technical background, he primarily expressed his needs in terms of how he wanted to interact with the system. This naturally shaped the UI and the workflow we designed for scholars. Additionally, the inclusion of segmentation-based machine learning came from his explicit request; he emphasized that automated assistance in identifying and matching fragments would significantly improve his research workflow. These inputs helped anchor our design choices in real user needs.

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

   Very few changes were needed to our previous documents. Because this is a pre-approved project with a well-defined problem and scope, many of the requirements and expectations were already laid out for us in the project description. As a result, our earlier documents, such as the SRS and Hazard Analysis, aligned naturally with

the design work we did here. We did not need to revisit or adjust them significantly, beyond minor wording refinements and consistency updates.

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

   With unlimited resources, one of the most impactful improvements would be investing in more powerful AI infrastructure, such as GPUs dedicated to training and evaluating models. This would allow us to train new segmentation or matching models more frequently and at a much larger scale. We would also aim to build a more advanced fragment-matching pipeline with higher accuracy and more robust behaviour across different manuscript types. Additionally, we would elevate the quality of the UI significantly, improving performance, adding more interactive tools, and making the interface cleaner and more intuitive. Overall, the entire system could operate at a higher level of precision, speed, and sophistication.

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

   We did not explore many alternative designs because, from early discussions, the entire team already shared a clear mental model of how the system should operate. Our frequent conversations kept us aligned on the major components and workflow, so we naturally converged on a single coherent design. Although this meant we did not formally evaluate multiple competing solutions, the advantage was that we were able to focus our efforts on developing one strong, detailed design rather than splitting time considering options that did not align with the client's expectations. Given the clarity of the problem and the constraints provided, the chosen design was the most straightforward and appropriate path forward.

Module Reflection

Implementing the User Authorization Module (M8) was largely a smooth process due to the clarity of the provided MIS structure, though there were specific areas where ambiguity slowed down development. On the positive side, the Semantics section clearly defined the necessary state variables, making it immediately obvious that we needed `activeSessions` and `tokenExpiryTimes` as dictionary structures to manage session tokens and expiration timestamps. Additionally, the Exported Access Programs table was highly effective; having a concise checklist of inputs, outputs, and exceptions allowed us to scaffold the function signatures for `login`, `logout`, and `validateToken` very quickly. The inclusion of Exported Constants like `TOKEN_EXPIRY_MIN` was also beneficial, as it removed the need for magic numbers in the code and clearly indicated which features, such as refresh tokens, needed to be configurable.

However, we encountered challenges regarding data types and external integration. While the interface listed types like `Credentials` and `UserProfile`, the MIS did not define their internal structure, forcing us to make assumptions about the required fields (e.g., whether `Credentials` included just a password or also an API key). Furthermore, while the module specified a reliance on an External Authentication Provider and an `AUTH_SERVER_URL`, it lacked the specific API contract or protocol details required to implement the actual communication with that provider. There was also minor confusion regarding `SystemTime` and whether it was an environment variable to be mocked or simply the system clock.

To improve the design documentation for future implementations, we recommend adding a dedicated "Data Types" subsection to explicitly define the fields of custom objects like `UserProfile` to prevent integration mismatch. It would also be helpful to include pseudocode or step-by-step semantic descriptions for complex operations like `login`, clarifying exactly when to update internal state versus calling external providers. Finally, adding a column for specific exception trigger conditions—defining precisely *why* a `TokenInvalid` exception should be thrown—would greatly aid in writing accurate unit tests.